Université Toulouse Capitole - TSE

Web Mining Project – 2025-2026

Mining Knowledge from Social Media Data During Crisis Events

Alexandre Picart & Simon Simus

March 27, 2026

Contents

1 Introduction **2**

2 Data Description and Preprocessing **2**
2.1 Dataset Overview . . . 2
2.2 Text Cleaning and Tokenization . . . 2

3 Part 1 : Implement a core search system on the tweet corpus **3**
3.1 Semantic Maps by Event Type . . . 3
3.1.1 Word-Embedding Clusters (Glove-Twitter-25) . . . 3
3.1.2 Tweet-Embedding Clusters (Sentence-Transformers) . . . 4
3.2 Question-Answering System: Two-Stage Retriever–Re-ranker . . . 5
3.2.1 Architecture . . . 5
3.2.2 Test Queries . . . 6
3.2.3 Evaluation Metrics and Results . . . 6
3.2.4 Analysis of Results . . . 6

4 Part 2 – Tweet Priority Prediction **7**
4.1 Data Splitting . . . 7
4.2 Feature Engineering . . . 7
4.2.1 Sentiment Feature (Model Improvement) . . . 7
4.3 Handling Class Imbalance . . . 7
4.4 Learning and Evaluation . . . 8
4.4.1 Results with SMOTE . . . 8
4.4.2 Cross-Validation Results . . . 8
4.5 Model Improvement: Hyperparameter Optimization . . . 9

5 Conclusion **10**

1 Introduction

The objective of this project is to analyse social media data (tweets from X) produced during past crisis events (floods, wildfires, earthquakes, bombings, etc.) from a web mining perspective. The dataset originates from the NIST TREC Incident Stream Initiative and contains real-world tweets posted during emergencies.

Our work addresses two complementary goals:

1. **Part 1 : Implement a core search system on the tweet corpus:** building semantic maps of the tweet corpus by event type (word-embedding clusters, tweet-embedding clusters) and implementing a Question-Answering (QA) system with a two-stage retriever re-ranker architecture.
2. **Part 2 – Priority prediction:** predicting the priority level of a tweet (High, Medium, or Low) using content, usage and structure features, to help emergency officers triage incoming information.

2 Data Description and Preprocessing

2.1 Dataset Overview

The cleaned dataset is structured as a graph database with nodes (Events, Tweets, Users, Hashtags, Post Categories) and relationships (Posted, Retweeted, Reply_To, Mentions, Has_Hashtag). All files are provided in JSON format.

After loading and flattening the nested JSON records with `pd.json_normalize`, we obtain a working dataframe of 55 986 tweets spanning 6 event types: *bombing*, *earthquake*, *flood*, *shooting*, *typhoon*, and *wildfire*. Each tweet is associated with an event via a TREC-IS topic identifier and carries a human-labelled priority among {Low, Medium, High, Critical}.

2.2 Text Cleaning and Tokenization

We apply the following preprocessing pipeline to each tweet:

1. Convert text to lowercase.
2. Remove URLs, user mentions (`@user`), and digits.
3. Strip the `#` symbol but we keep the hashtag text (example: #HighParkFire → highparkfire).
4. Remove remaining punctuation.
5. Tokenize with NLTK's word_tokenize.
6. Filter out english stop words (NLTK defaults plus an extra list: rt, amp, http, co, would, could, etc.).

The resulting tokens are joined back into a cleaned_text column used for embedding computation. Table 1 illustrates the effect of this pipeline on three sample tweets. Note that stop words are filtered later in another step.

#	Raw tweet	Tokens after cleaning
1	#Media Large wildfire in N. Colorado prompts evacuations: Crews are battling a fast-moving wildf... http://t.co/ju1BGTKH #Politics #News	['media', 'large', 'wildfire', 'in', 'n', 'colorado', 'prompts', 'evacuations', 'crews', 'are', 'battling', 'a', 'fast', 'moving', 'wildf', 'politics', 'news']
2	Large wildfire in northern Colorado prompts evacuations - Fox News: CBS NewsLarge wildfire in northern Colorado ... http://t.co/BvG6xSRH	['large', 'wildfire', 'in', 'northern', 'colorado', 'prompts', 'evacuations', 'fox', 'news', 'cbs', 'newslarge', 'wildfire', 'in', 'northern', 'colorado']
3	RT @LarimerSheriff: #HighParkFire update http://t.co/hBy5shen	['rt', 'highparkfire', 'update']

Table 1: Examples of the tokenization and cleaning pipeline.

3 Part 1 : Implement a core search system on the tweet corpus

3.1 Semantic Maps by Event Type

3.1.1 Word-Embedding Clusters (Glove-Twitter-25)

For each event type, we build a semantic map of the most characteristic vocabulary as follows:

1. Collect the top-50 most frequent valid words (alphabetic, length > 2, not in stop words, present in the GloVe vocabulary).
2. Retrieve their 25-dimensional GloVe-Twitter vectors (glove-twitter-25 is loaded via `gensim.downloader`).
3. Apply K-Means ($k = 5$) to partition the words into semantic clusters.
4. Reduce to 2D with PCA and produce a scatter plot where each point is a word, coloured by cluster membership.

The choice of Glove-Twitter over Word2Vec is because Glove Twitter was trained on tweets, making it better suited to capture the informal vocabulary of social media crisis communication.

Moreover we chose to keep only the top 50 most frequent valid word to have a clear graph.

Key observations:

- For *bombing* (see Figure 1), the five clusters clearly separate the distinct events and reaction types within this category. Cluster 4 groups terms related to the Paris attacks (paris, concert, bataclan, france. . .), while Cluster 0 is associated with the Boston Marathon bombing (bostonmarathon, prayforboston, waco. . .).
  Cluster 2 gathers factual terms mixing the Boston bombing with the West, Texas fertilizer plant explosion (explosion, police, bomb, fertilizer, plant. . .).
  Cluster 1 captures empathy and reaction vocabulary (prayers, thoughts, people, killed. . .), and Cluster 3 corresponds to human impact and hostage situations (hostage, victims, families. . .). This separation demonstrates that word-embedding clusters can effectively distinguish sub-events within a single event type.
- For *flood* (see Figure 2), the clusters separate geographic locations from flood-related vocabulary. One cluster groups place names from various flood events across different continents (calgary, alberta, queensland, manila, colombia. . .), while another isolates South African locations (durban, kwazulu, kzn. . .). A third cluster gathers weather and disaster terms

(rain, flooding, mudslides, water, heavy. . .), and the remaining clusters distinguish emergency response vocabulary (rescue, emergency, toll. . .) from general reaction and solidarity expressions (help, people, safe, need. . .).

- For *earthquake* (see Figure 1), the clusters reveal a clear geographic and thematic structure. Two clusters separate distinct earthquake locations: one groups Latin American and Southeast Asian place names (guatemala, chile, manila. . .), the other focuses on the Nepal earthquake (nepal, kathmandu, luzon. . .). A third cluster captures seismological vocabulary (earthquake, magnitude, tsunami. . .), while the remaining two distinguish humanitarian impact terms from emotional and solidarity expressions. As with bombing, the embedding space effectively separates geographic context from factual reporting and emotional responses.

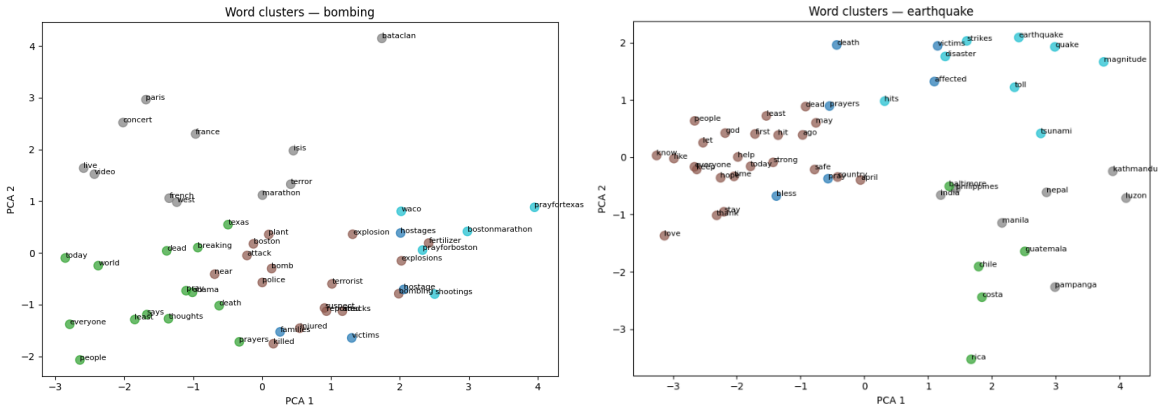

Figure 1: Word embedding clusters for the *bombing* and *earthquake* event types. Each point is a word, coloured by cluster assignment.

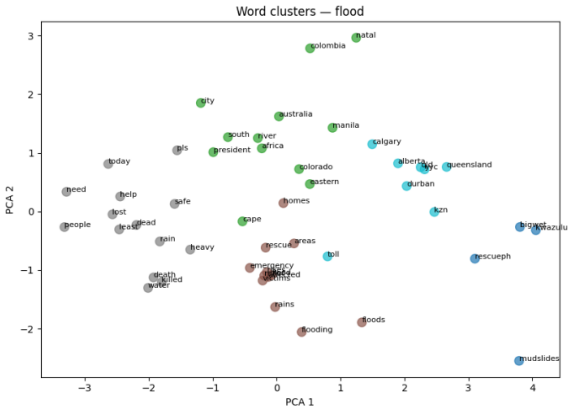

Figure 2: Word embedding clusters for the *flood* event types.

3.1.2 Tweet-Embedding Clusters (Sentence-Transformers)

To capture the semantics at the tweet level, we use the **all-MiniLM-L6-v2** Sentence-Transformer model, which encodes each tweet into a 384-dimensional dense vector. Unlike the word-embedding maps where PCA was sufficient to project 25 dimensions, the 384-dimensional tweet embeddings contain complex non-linear relationships that PCA cannot preserve; t-SNE is therefore preferred as it better captures local neighborhood structure in high-dimensional spaces.

The clustering procedure is:

1. Encode all tweets with the Sentence-Transformer.
2. For each event type, sample up to 500 tweets for clear visualization.
3. Apply K-Means ($k = 5$) on the sample.
4. Reduce to 2D with t-SNE (perplexity 30, PCA initialization) and produce a scatter plot.
5. Identify the representative tweet per cluster: the tweet closest to the cluster centroid.

The t-SNE visualizations reveal well-separated groups: for instance, in *bombing*, one cluster gathers tweets about the Boston Marathon bombing, another about the Paris attacks, and a third about the West, Texas fertilizer plant explosion. Similarly, in *shooting* (see Figure 3), the representative tweets show that each cluster corresponds to a distinct shooting event: the Dallas protest shooting, a San Diego synagogue shooting, the LAX airport shooting, and school shootings in Colorado, confirming that the model captures event-level semantics. Representative tweets confirm that the clusters capture coherent sub-events or information themes within each event type.

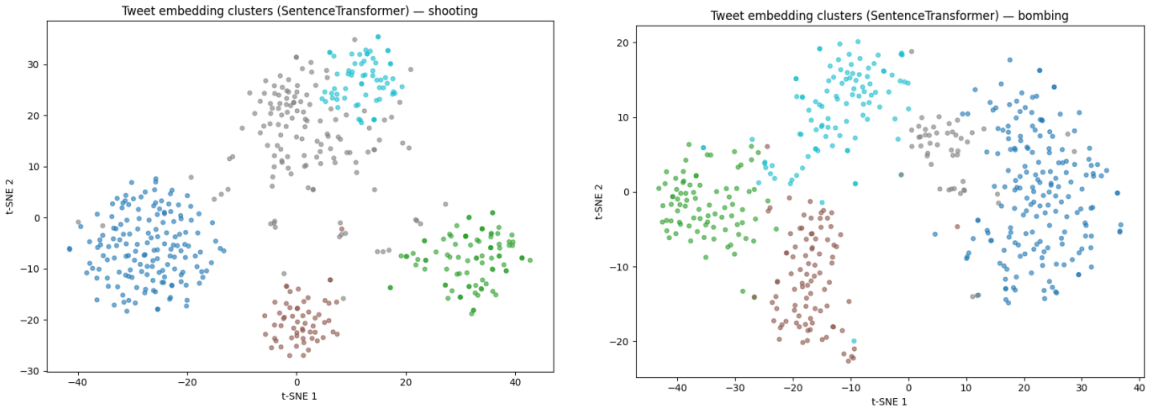

Figure 3: Tweet-embedding clusters for *bombing* and *shooting*.

Cluster	Representative tweet
0	Oh shit! Shooting going on in Dallas at protest. Apparently some cops shot. https://t.co/0pJHqSHWJv.
1	Shooting reported at San Diego-area synagogue https://t.co/COZnd0iD6l ...
2	RT @heatherhoylee: The shooting at LAX is going to make airport security even more of a pain in the ass ...
3	Another school shooting in America?
4	school shooting in Colorado, 2 injured.

Table 2: Representative tweet per cluster for the *shooting* event type. Each tweet is the closest to its cluster centroid in the Sentence-Transformer embedding space.

3.2 Question-Answering System: Two-Stage Retriever–Re-ranker

3.2.1 Architecture

We implement a two-stage pipeline:

1. Step 1 : Bi-encoder retrieval: all tweets are pre-encoded with `all-MiniLM-L6-v2`. Given a query, we encode it with the same model, compute cosine similarity against the full corpus, and retain the top-1 000 candidates.

2. Step 2 : Cross-encoder re-ranking: each (query, candidate tweet) pair is scored by `cross-encoder/ms-marc` a model fine-tuned on MS MARCO for passage re-ranking. We keep the **top-10 results.**

This architecture balances efficiency (the bi-encoder scales to the full 55k corpus) with accuracy (the cross-encoder provides fine-grained relevance scores on a manageable candidate set).

3.2.2 Test Queries

We construct a toy evaluation set of 4 queries, each paired with a known relevant tweet from the corpus . All queries focus on the High Park Fire wildfire event:

ID	Query
q1	Where is the evacuation center for the High Park Fire in Colorado?
q2	Did FEMA provide funds for the High Park Fire?
q3	How many acres has the High Park Fire burned and how many structures were damaged?
q4	Where can High Park Fire evacuees board their animals?

Table 3: Test queries for the QA system evaluation. Each query has one gold-standard tweet.

3.2.3 Evaluation Metrics and Results

We evaluate with three standard IR metrics: Precision@10 , Recall@10 and the Reciprocal Rank@10 (RR@10) that give us MRR when we take the mean of queries

Query	P@10	Recall@10	RR
q1	0.10	1.00	0.111
q2	0.10	1.00	1.000
q3	0.10	1.00	0.333
q4	0.10	1.00	1.000
Mean	**0.10**	**1.00**	**MRR = 0.611**

Table 4: Evaluation results for the two-stage retriever–re-ranker.

Note that we use RR here and not AP because when we have only 1 relevant document to find, RR and AP give the same result. The precision is not very interesting for the same reason.

3.2.4 Analysis of Results

Recall is perfect (1/1): the gold-standard tweet always appears in the top-10 re-ranked results for all 4 queries, meaning the pipeline successfully surfaces the relevant document.

Precision is low (0.10): this is expected since each query has only 1 gold tweet among 10 retrieved, giving $P@10 = 1/10 = 0.10$. This metric would be more informative with a larger gold set.

RR varies across queries:

- q2 and q4 (RR = 1.0): the gold tweet is ranked first. These queries contain specific, distinctive keywords ("FEMA funds", "board their animals") that align well with the bi-encoder and cross-encoder scoring.
- q3 (RR = 0.33): the gold tweet appears at rank 3. The query asks about multiple facts (acres burned + structures damaged), which dilutes the relevance signal.

- q1 (RR = 0.11): the gold tweet appears at rank 9. The query about "evacuation center" is more generic, matching many tweets about evacuations, so the specific tweet about Cache La Poudre Middle School is ranked lower.

However we can observe that when we don't have the gol tweet in first place, others tweets that are ranked first are well linked to the subjet.

4 Part 2 – Tweet Priority Prediction

Given a tweet, the goal is to predict its priority among three classes: *High*, *Medium*, and *Low*. We exclude the *Critical* class due to its very low frequency in the dataset.

4.1 Data Splitting

We split the dataset into 70% training and 30% testing with stratification to preserve the class distribution:

Priority	Count	Proportion
Low	25 407	70.2%
Medium	6 629	18.3%
High	4 155	11.5%

Table 5: Class distribution before splitting. The dataset is clearly imbalanced.

4.2 Feature Engineering

We combine features from the three facets of the social web graph:

- Content-based features: 384-dimensional Sentence-Transformer embedding of the tweet text (`all-MiniLM-L6-v2`).
- Usage-based features**:** presence of a hashtag (binary), tweet length (number of words).
- Structure-based featu**res:** retweet count and favorite count as proxies for tweet and user popularity.

All features are concatenated into a single matrix and standardized with `StandardScaler` to bring all dimensions to comparable scales.

4.2.1 Sentiment Feature (Model Improvement)

To improve the model, we add a sentiment score computed with SentiWordNet. For each word in the tweet, we determine its part-of-speech tag, look up the corresponding synset in WordNet, and compute the difference between positive and negative scores. The sum across all words gives a global sentiment score for the tweet.

The intuition is that crisis tweets with strong negative sentiment (expressing fear, urgency, or distress) tend to be associated with higher priority levels.

4.3 Handling Class Imbalance

The dataset is heavily imbalanced: *Low* priority tweets represent 70% of the data. We compare two strategies:

1. **SMOTE:** generates synthetic samples for the minority classes (*High* and *Medium*) in the training set, resulting in a perfectly balanced dataset of 17 784 samples per class.
2. **Cla**ss-weight balancing: passing `class_weight='balanced'` to scikit-learn estimators, which adjusts the loss function to penalize misclassification of minority classes.

Experiments show that SMOTE consistently produces better results than class-weight balancing for this dataset.

4.4 Learning and Evaluation

We train and compare three classifiers. The evaluation metric is the **macro-averaged F1 score** (treating all three classes equally) and the **Cohen's Kappa** coefficient (accounting for class imbalance and chance agreement).

4.4.1 Results with SMOTE

Model	F1 Macro	Cohen's Kappa
Logistic Regression	0.513	0.311
Random Forest	0.551	0.361
XGBoost	0.551	0.357

Table 6: Test set performance with SMOTE oversampling.

Observations:

- Random Forest and XGBoost outperform Logistic Regression by roughly 4 points in F1 macro.
- The relatively modest F1 scores reflect the inherent difficulty of the task: priority labels are subjective and the textual signal distinguishing *Medium* from *Low* is often subtle.
- Logistic Regression achieves reasonable recall for *High* (0.56) but at the cost of many false positives (precision 0.29).

4.4.2 Cross-Validation Results

To obtain more robust estimates, we perform 5-fold cross-validation on the SMOTE-balanced training set:

Model	Mean F1	Std F1	Mean Kappa	Std Kappa
Logistic Regression	0.625	0.007	0.437	0.010
Random Forest	0.907	0.002	0.861	0.004
XGBoost	0.880	0.002	0.821	0.004

Table 7: 5-fold cross-validation on the SMOTE-balanced training set. These scores should be interpreted with caution: SMOTE was applied before splitting into folds, meaning synthetic samples derived from the training data may leak into the validation folds, leading to overly optimistic estimates. This explains the gap between the high CV scores (e.g. F1 = 0.907 for Random Forest) and the more modest test set performance (F1 = 0.551). The correct approach would be to apply SMOTE inside each fold. Nevertheless, the relative ranking of models remains consistent: Random Forest and XGBoost outperform Logistic Regression.

4.5 Model Improvement: Hyperparameter Optimization

We use **Weights & Biases (wandb)** to perform Bayesian hyperparameter optimization on the XGBoost classifier. The search space includes:

- `n_estimators` $\in \{50, 100, 200, 300\}$: the number of boosting rounds. More trees can improve performance but increase training time and risk of overfitting.
- `max_depth` $\in \{3, 5, 7, 10\}$: maximum depth of each tree. Deeper trees capture more complex patterns but are more prone to overfitting.
- `learning_rate`: sampled uniformly between 0.01 and 0.3. Controls how much each tree contributes to the final prediction; lower values require more trees but often generalize better.
- `subsample`: sampled uniformly between 0.6 and 1.0. Fraction of training samples used per tree, introducing randomness to reduce overfitting.
- `colsample_bytree`: sampled uniformly between 0.6 and 1.0. Fraction of features used per tree, adding further regularization.

The optimization target is F1 macro. The Bayesian strategy (wandb sweep) explores the parameter space more efficiently than grid search by modelling the objective function and focusing on promising regions.

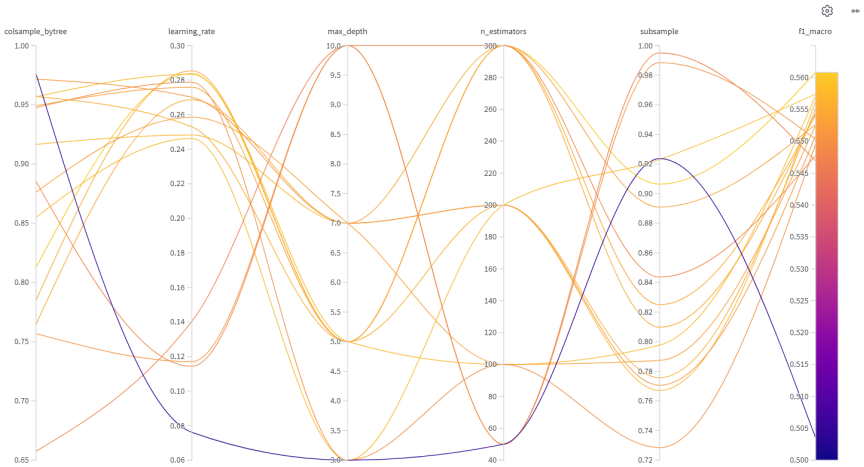

Figure 4: Weights & Biases sweep dashboard showing the Bayesian hyperparameter optimization runs for XGBoost. Each point is a configuration; the y-axis shows the F1 macro score.

The best configuration found by the sweep is presented in Table 8.

Hyperparameter	**Best value**
`n_estimators`	300
`max_depth`	5
`learning_rate`	0.284
`subsample`	0.906
`colsample_bytree`	0.813
F1 Macro	**0.561**

Table 8: Best XGBoost configuration found by Bayesian optimization (20 runs). The optimized model achieves F1 = 0.561, a modest improvement over the default configuration (F1 = 0.551).

5 Conclusion

In this project, we explored a crisis-tweet dataset through two complementary lenses.

In **Part 1**, semantic maps based on GloVe word embeddings and Sentence-Transformer tweet embeddings revealed coherent thematic clusters within each event type. The two-stage QA system (bi-encoder retrieval + cross-encoder re-ranking) achieved perfect recall on our test queries, with specific factual queries reaching MRR = 1.0. Performance degrades for broader queries, suggesting that event-specific filtering or query expansion could be beneficial.

In **Part 2**, we framed tweet priority prediction as a multi-class classification task. Random Forest and XGBoost outperformed Logistic Regression, reaching F1 macro scores around 0.55 on the test set. SMOTE proved more effective than class-weight balancing for handling the imbalanced dataset. The addition of a SentiWordNet-based sentiment feature and Bayesian hyperparameter optimization via Weights & Biases yielded a slight improvement (F1 = 0.561), suggesting that the performance ceiling is largely constrained by the inherent subjectivity of priority labels rather than by model capacity. Future work could integrate structural features such as graph embeddings via Node2Vec or user network centrality, and explore fine-tuned transformer-based classifiers to better capture the nuances of crisis communication.